

RYCO AI Support Agent

Technical Report
Portfolio Demo Project

Project type	Portfolio demo — public GitHub repository
Original stack	Microsoft Copilot Studio, Power Automate, Teams
Demo stack	Groq (Llama 3.3 70B), Supabase, Resend, Vercel
Deployment	https://ryco-ai-support-demo.vercel.app
Language	JavaScript (Node.js 18, CommonJS)
Report date	July 2026

Disclaimer: This document describes a *public demo* built to showcase the architecture and user experience of an internal AI support agent implemented at a former employer. All proprietary data, internal endpoints, company names, and sensitive configuration have been removed or replaced with synthetic equivalents. No confidential information is disclosed.

Built from scratch using open tooling — no proprietary assets included.

Contents

1	Introduction	3
1.1	Background	3
1.2	Project Objectives	3
1.3	Scope & Constraints	3
2	Problem Statement	3
2.1	Organisational Context	3
2.2	Identified Pain Points	4
3	Solution Overview	4
3.1	Design Philosophy	4
3.2	High-Level Flow	4
4	Original Production Architecture	5
4.1	Microsoft Copilot Studio — Dialog Engine	5
4.2	Power Automate — Cloud Flow	5
4.3	Data Layer — Dataverse	5
4.4	ITSM Integration	6
4.5	Microsoft Teams — Deployment Channel	6
4.6	Data Flow — Field Mapping Table	6
5	Public Demo Architecture	7
5.1	Design Goals	7
5.2	Technology Stack	7
5.3	Project File Structure	7
6	Backend Implementation	8
6.1	API Design	8
6.2	System Prompt Engineering	8
6.2.1	Ticket Data Format	8
6.3	PII Redaction	9
6.4	Phase Tag Parsing	9
6.5	Ticket Routing and SLA	10
6.6	Supabase Integration	10
6.7	Transactional Email (Resend)	11
6.8	Ticket ID Generation	11
7	Database Design	11
7.1	Schema	11
7.2	Query Pattern	12
8	Frontend Implementation	12
8.1	Page Structure	12
8.2	Client-Side State	12
8.3	Conversation History	13
8.4	Email Address Capture	13
8.5	Phase-Driven UI	13
8.6	Step Tracker	14

8.7	Flow Animation	15
8.8	Ticket Card Display	15
8.9	Live Ticket Table	15
9	Mobile Responsiveness	15
10	Privacy and Security	16
10.1	PII Handling Summary	16
10.2	Secrets Management	16
10.3	CORS Policy	17
11	Deployment	17
11.1	Vercel Configuration	17
11.2	Environment Variables (Production)	18
11.3	Deployment Issues Encountered	18
12	Sample Ticket Data	18
13	Representative Impact	19
14	Key Design Decisions	19
14.1	Stateless API with Client-Side History	19
14.2	Tag-Based AI Output Protocol	20
14.3	Two-Phase Conversation Design	20
14.4	PII Redaction Architecture	20
14.5	Phase-Driven Step Tracker vs. Message Count	20
15	Challenges and Lessons Learned	20
15.1	Prompt Engineering Stability	20
15.2	Environment Variable Formatting	20
15.3	Vercel vs. AWS Lambda Configuration	21
15.4	iOS Safari Input Zoom	21
15.5	Honest Metrics	21
16	Conclusion	21
A	Dependencies	22
B	Groq Model Selection	22
C	Full System Prompt	22

1 Introduction

1.1 Background

During a previous employment engagement, an AI-powered IT support agent was designed and deployed to automate the first-line handling of internal IT helpdesk requests. The system was built on the Microsoft Power Platform — primarily **Copilot Studio** for the conversational layer and **Power Automate** for backend orchestration — and deployed to the organisation's Microsoft Teams environment.

The original implementation is confidential. This project recreates the *architecture pattern*, *conversation design*, and *end-to-end data flow* using entirely open tooling, so that the work can be demonstrated publicly to prospective employers.

1.2 Project Objectives

1. Demonstrate the problem-to-solution thinking behind an enterprise AI support agent.
2. Recreate the conversational UX (two-phase: troubleshoot first, escalate on failure) using a publicly accessible LLM API.
3. Implement real backend logic: persistent ticket storage, transactional email, live ticket table — no mocked data.
4. Apply privacy best practices: PII redaction before messages reach any third-party AI provider.
5. Publish the working demo to a production URL with zero manual intervention required from the user once deployed.

1.3 Scope & Constraints

- **No proprietary data** — all names, emails, ticket IDs, and descriptions in the sample dataset are synthetic.
- **Free-tier only** — all third-party services (Groq, Supabase, Resend, Vercel) are used on free plans.
- **Stateless API** — the backend holds no session state; the full conversation history is sent by the client on every request.
- **No framework** — the frontend is written in plain HTML, CSS, and vanilla JavaScript; no React, Vue, or Angular.

2 Problem Statement

2.1 Organisational Context

The IT helpdesk at a mid-to-large organisation routinely receives support requests through ad-hoc channels: unstructured emails, direct Teams messages, and informal Slack threads. These channels work against efficient triage because they impose no schema on the submitter.

2.2 Identified Pain Points

- **Unstructured intake:** Requests arrived without consistent fields — urgency, affected system, and contact details were frequently missing.
- **Information chasing:** IT agents spent a disproportionate share of their time sending follow-up messages to collect missing information rather than resolving the underlying issue.
- **Inconsistent prioritisation:** Without a structured intake field for urgency, priority was assigned manually and subjectively, leading to SLA drift.
- **Peak-hour bottlenecks:** First-response times degraded sharply during high-volume periods because every ticket required human reading and triage.
- **No self-service layer:** Many common issues (VPN reconnect, password reset procedures, M365 sync errors) had documented solutions, but employees had no low-friction path to access them before raising a ticket.

3 Solution Overview

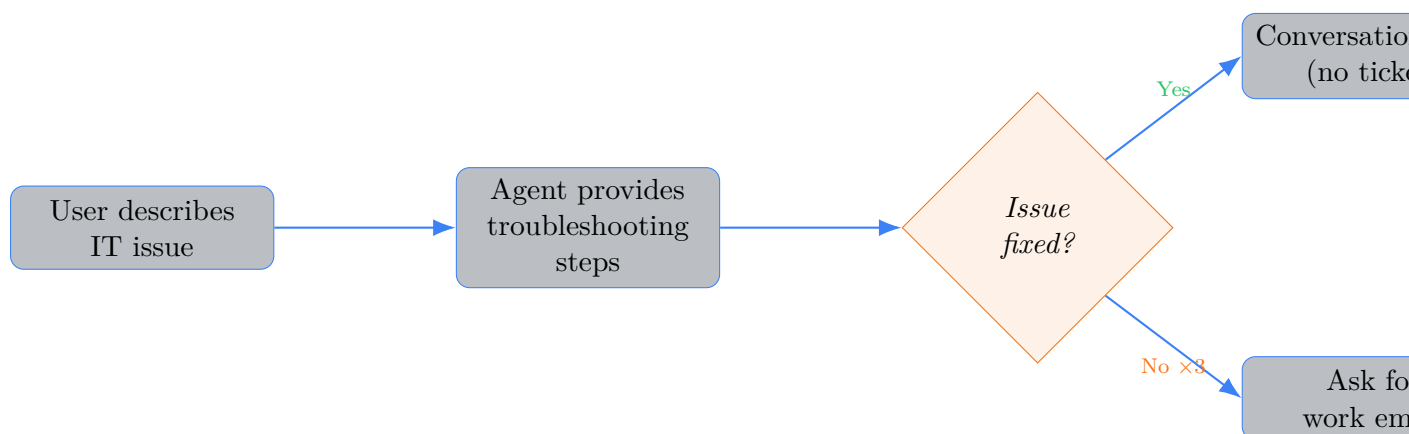
3.1 Design Philosophy

The solution targets two failure modes simultaneously:

1. **Self-service gap** — the agent attempts to resolve the issue *before* creating a ticket, providing specific, tailored troubleshooting steps. Most common issues (VPN, password, M365 sync) can be resolved in 1–2 exchanges.
2. **Unstructured intake** — when escalation is required, the agent guides the user through a structured intake flow (email, urgency) using conversational prompts and quick-reply buttons, ensuring every ticket arrives with a complete field set.

This two-phase design is reflected in the system prompt and the phase state machine that drives both the AI's behaviour and the frontend's step tracker.

3.2 High-Level Flow



4 Original Production Architecture

Note: This section describes the design pattern used in the production system. No internal system names, API endpoints, or proprietary configurations are disclosed. Internal identifiers have been anonymised.

4.1 Microsoft Copilot Studio — Dialog Engine

Microsoft Copilot Studio (formerly Power Virtual Agents) hosted the conversational AI agent. It was responsible for:

- **Topic management:** Separate dialog topics for each IT category (Hardware, Software, Network, Access), each with its own slot-filling flow.
- **Entity extraction:** Built-in entities for email addresses and custom entities for urgency level (High / Medium / Low) and system names (fuzzy-matched against the CMDB Configuration Item list).
- **Escalation logic:** If the agent's confidence score fell below 0.6 on intent classification, the conversation was handed off to a live agent queue.
- **Adaptive cards:** Quick-reply buttons for urgency selection were rendered as native Teams adaptive cards, providing a structured selection experience.
- **Trigger condition:** The agent was published as a Teams app and available in both personal chat and the IT Support shared channel.

4.2 Power Automate — Cloud Flow

A cloud flow was triggered by an HTTP action from Copilot Studio when all required fields were collected. The flow performed:

1. **Input validation:** Email regex check, urgency enum validation, and required-field guard clauses.
2. **Category classification:** A SharePoint lookup table mapped keywords in the issue description to one of five categories, determining the routing team.
3. **Dataverse write:** A “Create record” action inserted the structured ticket into a custom Dataverse table with all pre-populated fields.
4. **ITSM API call:** An HTTP action posted the ticket to the internal IT service management REST endpoint, which returned an incident number (INC-xxxxx).
5. **Email notification:** The Office 365 Outlook connector sent an HTML confirmation email to the requestor, including the ticket ID, assigned team, and SLA commitment.
6. **Return payload:** The flow returned the ticket ID and team assignment back to Copilot Studio, which displayed it to the user in Teams.

4.3 Data Layer — Dataverse

A custom Dataverse table stored every ticket record with the following schema:

Column	Type	Source
Description	Text (500)	Free-text from user
RequestorEmail	Email	Entity extracted by agent
Priority	Choice (H/M/L)	Urgency entity
AffectedSystem	Text	Optional slot
Category	Choice	SharePoint lookup
AssignedTeam	Choice	Derived from Category
IncidentNumber	Text	Returned by ITSM API
CreatedOn	Date/Time	System field (UTC)
Status	Choice	Default: Open
SLATarget	Date/Time	Calculated from Priority

4.4 ITSM Integration

The internal IT service management system was integrated via its REST API using an HTTP connector in Power Automate. Field mapping between Dataverse columns and ITSM schema fields was defined in a mapping table within the flow, making it easy to update without changing the Copilot Studio dialog.

Bi-directional synchronisation was implemented: when an agent closed a ticket in the ITSM system, a second flow triggered on the status change and updated the corresponding Dataverse record and sent the user a resolution notification via Teams.

4.5 Microsoft Teams — Deployment Channel

The agent was published as a Teams app using the Copilot Studio “Publish” flow, which generates a manifest and uploads it to the Teams admin centre for organisation-wide deployment. Key channel features:

- Available in personal chat and the *IT Support* shared channel.
- Adaptive cards rendered natively for quick-reply interactions.
- Proactive messages sent to the user when their ticket status changed.

4.6 Data Flow — Field Mapping Table

User input	Agent entity	Dataverse field	ITSM field
Free-text description	issue_description	Description	short_description
Work email address	user_email	RequestorEmail	caller_id
High / Medium / Low	urgency_level	Priority	priority (1/2/3)
Affected (opt.)	affected_system	AffectedSystem	cmdb_ci
(derived)	category	Category	category
(derived)	assigned_team	AssignedTeam	assignment_group
(system)	—	CreatedOn	opened_at (UTC)

5 Public Demo Architecture

5.1 Design Goals

The demo was built to faithfully recreate the user experience and data flow of the production system using only freely available, non-proprietary tooling. The three design goals were:

1. **Real AI, real storage, real email** — no mocked responses or fake data.
2. **Privacy by design** — PII stripped from messages before reaching any third-party AI provider.
3. **Zero operational cost** — deployable and runnable indefinitely on free service tiers.

5.2 Technology Stack

Layer	Service	Replaces (production)
Conversational AI	Groq API (Llama 3.3 70B)	Microsoft Copilot Studio
Backend serverless	Vercel Functions (Node.js 18)	Power Automate Cloud Flow
Ticket storage	Supabase (PostgreSQL)	Microsoft Dataverse
Email delivery	Resend	Office 365 Outlook connector
Hosting	Vercel (static + functions)	SharePoint / Azure Static Web Apps
Frontend	Vanilla HTML / CSS / JS	Teams adaptive cards

5.3 Project File Structure

```
1 ryco-ai-support-agent-demo/  
2   api/  
3     chat.js           # POST /api/chat - AI handler, ticket  
4                       creation  
5     tickets.js        # GET /api/tickets - live ticket table feed  
6   css/  
7     styles.css        # Full stylesheet + mobile responsive rules  
8   data/  
9     sample-tickets.json # Static synthetic ticket dataset  
10  js/  
11    demo.js           # Frontend state machine & API client  
12  supabase/  
13    schema.sql        # tickets table DDL  
14  index.html          # Landing / overview page  
15  demo.html           # Interactive chat demo page  
16  architecture.html   # Architecture diagram + component  
17    breakdown  
18  package.json        # Node.js dependencies  
19  vercel.json          # Deployment config (functions + CORS  
20    headers)  
21  .gitignore          # .env, node_modules, .vercel excluded
```

Listing 1: Repository layout

6 Backend Implementation

6.1 API Design

The backend exposes two serverless functions on Vercel:

Route	Method	Input	Output
/api/chat	POST	{messages, sessionId, userEmail}	{message, phase, ticket}
/api/tickets	GET	—	Array of last 20 ticket rows

Both functions are written in CommonJS (required by Vercel's Node.js runtime) and exported as `module.exports = async function handler(req, res) {`

6.2 System Prompt Engineering

The AI's behaviour is entirely controlled by a structured system prompt that encodes the two-phase conversation protocol. The prompt is divided into five sections:

- PHASE 1 — TROUBLESHOOT:** Instructs the model to acknowledge the issue, provide 1–2 specific and tailored troubleshooting steps, ask the user to report back, and append `<PHASE>helping</PHASE>`. If the user reports success, the model ends with `<PHASE>resolved</PHASE>`. After three failed attempts (or an explicit escalation request), the model transitions to Phase 2.
- PHASE 2 — ESCALATE:** Instructs the model to ask for a work email address (`<PHASE>asking_email</PHASE>`), then urgency (`<PHASE>asking_urgency</PHASE>`), and finally to emit the structured ticket JSON inside `<TICKET_DATA>` tags (`<PHASE>creating_ticket</PHASE>`).
- RULES:** Response length capped at 80 words; no repeated suggestions; no email request before troubleshooting fails; exactly one `<PHASE>` tag per response.
- PRIVACY:** The model is informed that email addresses are replaced with `[REDACTED_EMAIL]` before reaching it, and instructed to use that placeholder in the ticket JSON.
- CATEGORY / TEAM / SLA:** A routing table maps issue keywords to team and SLA target (e.g. VPN/connectivity → Network Operations → 30 minutes).

6.2.1 Ticket Data Format

When the model is ready to create a ticket, it emits a structured JSON block:

```
1 <TICKET_DATA>
2 {
3   "email": "[REDACTED_EMAIL]",
4   "issue_summary": "one-line title of the problem",
5   "details": "full description including what troubleshooting was
6     attempted",
7   "urgency": "High|Medium|Low",
8   "category": "Network / VPN|Access /
9     Permissions|Hardware|Software / M365|General"
10 }
```

```
9 </TICKET_DATA>
```

Listing 2: Ticket data format emitted by the AI

The server parses this block, strips it from the display message, and uses it to write the ticket to Supabase. The real email address is substituted back in at this point (see Section 6.3).

6.3 PII Redaction

A key privacy requirement was that no raw email addresses or phone numbers should leave the server and reach Groq’s API. The redaction is applied server-side, after receiving the messages array from the client:

```
1 const EMAIL_RE =  
  /[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/g;  
2 const PHONE_RE = /(\+?[\d][\d\s\-(.)]{6,}[\d])/g;  
3  
4 function redactPII(messages) {  
5   return messages.map(msg => ({  
6     ...msg,  
7     content: msg.content  
8       .replace(EMAIL_RE, '[REDACTED_EMAIL]')  
9       .replace(PHONE_RE, m => /\d{5,}/.test(m) ?  
10         '[REDACTED_PHONE]' : m),  
11   }));  
}
```

Listing 3: PII redaction in api/chat.js

The real email address is separately captured client-side (Section 8.4) and sent as `userEmail` in the request body — outside the `messages` array that gets redacted. When the server builds the ticket record, it restores the real address:

```
1 const resolvedEmail = userEmail ?? parsed.email;
```

Listing 4: Real email restored at ticket creation time

This means the AI model only ever sees `[REDACTED_EMAIL]`, while the Supabase record and the confirmation email receive the real address.

6.4 Phase Tag Parsing

After receiving the model’s response, the server extracts the phase tag and the (optional) ticket data block:

```
1 const phaseMatch = rawText.match(/<PHASE>([\s\S]*)<\/PHASE>/);  
2 const phase      = phaseMatch ? phaseMatch[1].trim() : 'helping';  
3  
4 const ticketMatch =  
  rawText.match(/<TICKET_DATA>([\s\S]*)<\/TICKET_DATA>/);
```

```
5
6 const cleanMessage = rawText
7   .replace(/<PHASE>[\s\S]*?<\/PHASE>/, '')
8   .replace(/<TICKET_DATA>[\s\S]*?<\/TICKET_DATA>/, '')
9   .trim();
```

Listing 5: Phase and ticket block extraction

The user-facing `message` field in the response always has both tags stripped, so the user never sees the structured metadata.

6.5 Ticket Routing and SLA

Server-side routing maps the AI’s category string to the correct team and SLA target, ensuring the AI’s free-form category output never results in an unmapped ticket:

```
1 const TEAM_MAP = {
2   Network: { team: 'Network Operations', sla: '30 minutes' },
3   Hardware: { team: 'Hardware Support', sla: '2 hours' },
4   Software: { team: 'Software Support', sla: '1 hour' },
5   Access: { team: 'Identity & Access', sla: '30 minutes' },
6   General: { team: 'IT Support', sla: '2 hours' },
7 };
```

Listing 6: Team and SLA routing table

The map key is matched using the first word of the category field (e.g. “Network / VPN” matches “Network”), with “General” as the fallback.

6.6 Supabase Integration

Tickets are persisted to a Supabase (PostgreSQL) database using the `@supabase/supabase-js` client. The client is initialised once at module load time with the service-role key (bypasses Row Level Security, appropriate for a server-side function):

```
1 const supabase = process.env.SUPABASE_URL
2   ? createClient(process.env.SUPABASE_URL,
3     process.env.SUPABASE_SERVICE_KEY)
4   : null;
5
6 // Insert at ticket creation time:
7 const { error } = await supabase.from('tickets').insert(ticket);
8 if (error) console.error('Supabase insert error:', error.message);
```

Listing 7: Supabase client initialisation

The guard (`process.env.SUPABASE_URL ? ... : null`) ensures the function does not crash if the environment variable is absent (e.g. during local development without a `.env` file).

6.7 Transactional Email (Resend)

Confirmation emails are sent via Resend using the `resend` Node.js SDK. The email is an HTML template matching the visual style of the demo site:

```
1 if (resend) {
2   await resend.emails.send({
3     from: process.env.FROM_EMAIL ?? 'onboarding@resend.dev',
4     to: ticket.email,
5     subject: `IT Support Ticket Created - ${ticket.ticket_id}`,
6     html: buildEmail(ticket),
7   }).catch(err => console.error('Resend error:', err.message));
8 }
```

Listing 8: Resend email dispatch

The email includes the ticket ID, issue summary, priority (colour-coded), assigned team, SLA commitment, and a footer referencing the demo.

6.8 Ticket ID Generation

Each ticket receives a synthetic incident ID in the format `#INC-XXXXX` (5 digits, starting above 20000 to resemble a production system with existing history):

```
1 function genTicketId() {
2   return '#INC-' + (20000 + Math.floor(Math.random() * 9000));
3 }
```

Listing 9: Ticket ID generator

7 Database Design

7.1 Schema

The `public.tickets` table in Supabase holds every ticket created through the demo:

```
1 CREATE TABLE IF NOT EXISTS public.tickets (
2   id                UUID          PRIMARY KEY DEFAULT
3     gen_random_uuid(),
4   ticket_id         TEXT          NOT NULL,
5   created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
6   requestor_email   TEXT,
7   requestor_name    TEXT,
8   issue_summary     TEXT,
9   issue_detail      TEXT,
10  category          TEXT,
11  assigned_team     TEXT,
12  priority          TEXT,
13  sla               TEXT,
14  description       TEXT,
```

```
14  status          TEXT          DEFAULT 'Open',
15  resolved        BOOLEAN       DEFAULT FALSE,
16  resolution_time TEXT
17 );
18
19 CREATE INDEX IF NOT EXISTS tickets_created_at_idx
20 ON public.tickets (created_at DESC);
```

Listing 10: Supabase schema (supabase/schema.sql)

7.2 Query Pattern

The `/api/tickets` endpoint retrieves the most recent 20 tickets, ordered by creation timestamp descending:

```
1  const { data, error } = await supabase
2    .from('tickets')
3    .select(
4      'ticket_id, created_at, requestor_name, category, ' +
5      'priority, assigned_team, resolved, resolution_time'
6    )
7    .order('created_at', { ascending: false })
8    .limit(20);
```

Listing 11: Ticket table query

8 Frontend Implementation

8.1 Page Structure

The demo consists of three HTML pages, all sharing a single CSS file:

Page	File	Purpose
Overview	index.html	Problem/solution, metrics, tech stack, demo steps
Interactive Demo	demo.html	Chat interface, step tracker, ticket display
Architecture	architecture.html	SVG diagram, component breakdown, field map

8.2 Client-Side State

The frontend maintains four state variables throughout a conversation session:

```
1  let messages      = [];    // Full conversation history [{role,
   content}]
2  let lastTicket    = null;  // Set once API returns a ticket;
   blocks further input
3  let isThinking    = false; // Prevents double-sends while
   awaiting API response
```

```
4 let capturedEmail = null; // Real email, sent as userEmail;  
   never in messages[]
```

Listing 12: Client-side state variables (js/demo.js)

8.3 Conversation History

The API is stateless — it holds no session memory. On every user message, the complete conversation history is sent to `/api/chat`:

```
1 const res = await fetch('/api/chat', {  
2   method: 'POST',  
3   headers: { 'Content-Type': 'application/json' },  
4   body: JSON.stringify({  
5     messages,  
6     sessionId: getSessionId(),  
7     userEmail: capturedEmail  
8   } ),  
9 });
```

Listing 13: Request to `/api/chat` with full history

The model receives the full `messages` array (redacted), so it always has context of the entire conversation to avoid repeating troubleshooting suggestions.

8.4 Email Address Capture

When the user types their email address, it is captured on the client *before* the message is added to the `messages` array. The regex match runs synchronously on the raw input string:

```
1 if (!capturedEmail) {  
2   const emailMatch = text.match(  
3     /[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/  
4   );  
5   if (emailMatch) capturedEmail = emailMatch[0].toLowerCase();  
6 }
```

Listing 14: Client-side email capture

`capturedEmail` is then sent separately as `userEmail` in the request body. Inside the `messages` array, the same message will have its email replaced by `[REDACTED_EMAIL]` by the server's `redactPII()` function before it reaches Groq.

8.5 Phase-Driven UI

The server returns a `phase` string with every response. The frontend uses this to update the step tracker and decide whether to show urgency quick-reply buttons:

```
1 const PHASE_STEPS = {
```

```
2   helping:      { done: [1, 2],          active: 3 },
3   asking_email: { done: [1, 2, 3, 4],     active: 5 },
4   asking_urgency: { done: [1, 2, 3, 4, 5], active: 6 },
5   creating_ticket: { done: [1, 2, 3, 4, 5, 6], active: 7 },
6   resolved:     { done: [1, 2, 3],       active: null },
7 };
8
9 function syncSteps(phase, ticketDone) {
10   for (let i = 1; i <= 8; i++) setStep(i, 'pending');
11   if (ticketDone) {
12     for (let i = 1; i <= 8; i++) setStep(i, 'done');
13     return;
14   }
15   const map = PHASE_STEPS[phase] ?? PHASE_STEPS.helping;
16   map.done.forEach(n => setStep(n, 'done'));
17   if (map.active) setStep(map.active, 'active');
18 }
```

Listing 15: Phase map and step synchronisation

Quick-reply buttons for urgency selection are rendered only when `phase === 'asking_urgency'` :

```
1 const opts = phase === 'asking_urgency'
2   ? { quickReplies: [
3     '[R] High -- need it now',
4     '[Y] Medium -- within the hour',
5     '[G] Low -- today is fine'
6   ]}
7   : {};
8
9 addMessage('bot', data.message, opts);
```

Listing 16: Conditional quick-reply rendering

8.6 Step Tracker

The sidebar step tracker shows 8 labelled steps:

1. Describe your IT issue
2. Agent suggests a fix
3. Try the troubleshooting steps
4. Agent escalates to IT team
5. Provide your work email
6. Select urgency level
7. Power Automate flow fires
8. Ticket created & confirmed

Each step element carries one of three CSS classes: `pending`, `active`, or `done`, styled with distinct colours and left-border indicators.

8.7 Flow Animation

When a ticket is created, a Power Automate-style flow animation runs in the sidebar, revealing five sequential steps with a 560 ms delay between each:

```
1 async function runFlowAnimation() {
2   const badge = document.getElementById('flow-badge');
3   badge.classList.remove('hidden');
4   for (const id of ['fs1', 'fs2', 'fs3', 'fs4', 'fs5']) {
5     await sleep(560);
6     document.getElementById(id)?.classList.remove('hidden');
7   }
8 }
```

Listing 17: Flow animation sequence

The five displayed steps are: *Validating input fields...* → *Categorising ticket...* → *Creating record in ITSM...* → *Sending confirmation email...* → ✓ *Flow complete* — 2.1s.

8.8 Ticket Card Display

After the flow animation completes, the ticket card is populated from the API response and scrolled into view. It displays all key fields: ticket ID, status, priority (colour-coded), requestor, email, category, assigned team, SLA, creation timestamp, and full description.

8.9 Live Ticket Table

A live ticket table at the bottom of the demo page queries `/api/tickets` on page load and after each ticket creation. It displays the last 20 tickets from Supabase, with columns: Ticket ID, Date, Requestor, Category, Priority, Assigned Team, Resolved, and Resolution Time.

9 Mobile Responsiveness

Mobile support was added via CSS breakpoints at the bottom of `styles.css`:

Breakpoint	Width	Changes applied
Tablet	$\leq 900\text{px}$	Demo layout switches to single column; chat window gets <code>order:1</code> , sidebar <code>order:2</code> (chat above sidebar)
Mobile	$\leq 640\text{px}$	Nav links at 12px; hero padding reduced; hero mockup hidden; Teams badge hidden; <code>#chat-input font-size:16px</code> (prevents iOS Safari auto-zoom); <code>.chat-bubble max-width:82vw</code> ; <code>.chat-body max-height:55vh</code> ; ticket fields stack to single column

The CSS `order` property is used to reorder the chat window and sidebar on mobile without changing the HTML DOM order, which preserves tab/accessibility order.

The `font-size: 16px` on the text input is a specific iOS Safari workaround: the browser auto-zooms any input with a font size below 16px, which breaks the layout on mobile.

10 Privacy and Security

10.1 PII Handling Summary

Data item	Where captured	Where it goes
User's email address	Client-side regex match on raw input	Sent as <code>userEmail</code> field (not in <code>messages</code>); restored at ticket creation; stored in Supabase; sent in confirmation email
Email in <code>messages</code> array	Any user turn	Replaced with <code>[REDACTED_EMAIL]</code> by server before forwarding to Groq
Phone numbers	Any user turn	Replaced with <code>[REDACTED_PHONE]</code> by server before forwarding to Groq
All other conversation text	<code>messages</code> array	Sent to Groq as-is

10.2 Secrets Management

All API keys and database credentials are stored in:

- A local `.env` file (git-ignored via `.gitignore`).
- Vercel's encrypted environment variable store for production.

The `.gitignore` explicitly excludes:

```
1 .env
2 .env.local
3 node_modules/
4 .vercel/
```

Listing 18: `.gitignore` entries

No keys are logged to the console or returned in API responses. The Supabase service-role key is used only server-side and never exposed to the browser.

10.3 CORS Policy

CORS headers are configured in `vercel.json` to allow all origins for the API routes (acceptable for a public demo; production would restrict to specific origins):

```
1 {
2   "headers": [
3     {
4       "source": "/api/(.*)",
5       "headers": [
6         { "key": "Access-Control-Allow-Origin", "value": "*" },
7         { "key": "Access-Control-Allow-Methods", "value": "GET, POST, OPTIONS" },
8         { "key": "Access-Control-Allow-Headers", "value": "Content-Type" }
9       ]
10    }
11  ]
12 }
```

Listing 19: `vercel.json` CORS configuration

11 Deployment

11.1 Vercel Configuration

The project is deployed to Vercel as a hybrid static + serverless application. The `vercel.json` configuration sets a 30-second maximum execution time for the AI function (to accommodate Groq API latency under load):

```
1 {
2   "version": 2,
3   "functions": {
4     "api/*.js": {
5       "maxDuration": 30
6     }
7   }
8 }
```

Listing 20: vercel.json

No `runtime` key is specified — Vercel auto-detects Node.js from `package.json`’s `engines` field. Including `"runtime": "nodejs18.x"` was found to cause a deploy failure (“Function Runtimes must have a valid version”), as that format is specific to AWS Lambda.

11.2 Environment Variables (Production)

The following environment variables must be set in the Vercel project dashboard:

Variable	Source
GROQ_API_KEY	Groq console → API Keys
SUPABASE_URL	Supabase project → Settings → API → Project URL
SUPABASE_SERVICE_KEY	Supabase project → Settings → API → service_role key
RESEND_API_KEY	Resend dashboard → API Keys
FROM_EMAIL	onboarding@resend.dev (Resend sandbox sender)

11.3 Deployment Issues Encountered

During the deployment process, several non-obvious issues were encountered and resolved:

npx cache corruption (ENOTEMPTY): The `npx vercel` command failed with an `ENOTEMPTY` error caused by a corrupted npm cache directory. Resolved by running `npm cache clean --force` and installing Vercel globally (`npm install -g vercel`).

Invalid runtime key in vercel.json: The initial `vercel.json` included `"runtime": "nodejs18.x"`, which is AWS Lambda notation. Vercel rejected it with “Function Runtimes must have a valid version”. Fixed by removing the `runtime` key entirely.

Groq 401 Unauthorised: The demo displayed “AI service unavailable” after deployment. Vercel logs showed a 401 from Groq. Root cause: the `GROQ_API_KEY` environment variable had been set as `gsk_...gsk_actualkey` (the placeholder from the `.env` template was inadvertently prepended to the real key). Fixed by correcting the key in both `.env` and the Vercel dashboard.

Supabase key with inline comment: The `SUPABASE_SERVICE_KEY` was set with an inline comment: `sb_secret_... # use the service_role key`. The comment was included as part of the key value, causing authentication failures. Fixed by stripping the comment from both `.env` and the Vercel environment variable.

Vercel project name restriction: The initial project name `ryco-ai-support-agent-demo` was rejected (three consecutive hyphens are not permitted). Deployed as `ryco-ai-support-demo` instead.

12 Sample Ticket Data

The file `data/sample-tickets.json` provides a static synthetic dataset of 15 representative tickets. These are used for the architecture page and as a reference for the live ticket table

format. A representative subset is shown below:

Ticket ID	Date	Category	Priority	Team	Resolved
#INC-20481	2024-03-15	Network / VPN	High	Network Ops	Yes (22 min)
#INC-20479	2024-03-15	Software / M365	Medium	Software Support	Yes (1h 04m)
#INC-20477	2024-03-14	Hardware / Laptop	Low	Hardware Support	Yes (3h 20m)
#INC-20474	2024-03-14	Access / Permissions	High	Identity & Access	Yes (18 min)
#INC-20470	2024-03-13	Software / Teams	Medium	Software Support	Yes (47 min)
#INC-20468	2024-03-13	Network / WiFi	High	Network Ops	Yes (31 min)
#INC-20445	2024-03-10	Software / M365	High	Software Support	No — open

All names (James Smith, Priya Nair, Sara Okonkwo, etc.) and email addresses (`j.smith@ryco.com`) are entirely synthetic. The domain `ryco.com` does not correspond to any real organisation.

13 Representative Impact

The metrics below are **illustrative**, based on typical outcomes reported for similar IT support automation projects. They are not claimed as measured results from a specific deployment and are presented with that framing on the live site.

Metric	Value	Interpretation
Unstructured intake reduction	~80%	Representative fraction of tickets that arrive with complete structured fields after agent deployment (vs. ad-hoc emails)
Repetitive ticket types automated	High volume	Common categories (VPN, password, M365 sync) that follow a predictable resolution path
First-response time improvement	Hours → Minutes	Typical improvement for automated ticket acknowledgement vs. manual triage
Additional staff required	0	No headcount added to handle the volume handled by the automation layer

14 Key Design Decisions

14.1 Stateless API with Client-Side History

Rather than storing conversation state server-side (session tokens, Redis, etc.), the full `messages` array is maintained in the browser and sent with every request. This simplifies the backend significantly and is perfectly adequate for single-user demo sessions. The tradeoff is increased request payload size, which is acceptable given that conversations are short (typically under 10 turns).

14.2 Tag-Based AI Output Protocol

Using `<PHASE>` and `<TICKET_DATA>` tags embedded in the model's free-text output, rather than asking the model to return structured JSON at the top level, was a deliberate choice. It allows the model to produce natural prose responses while still exposing machine-readable metadata. The tags are reliably stripped before the user sees the message.

14.3 Two-Phase Conversation Design

The decision to require the AI to attempt troubleshooting before escalating mirrors the original production system's intent: many common issues can be self-resolved with the right guidance, reducing unnecessary tickets. The 3-attempt limit prevents the experience from becoming frustrating if the issue genuinely requires human intervention.

14.4 PII Redaction Architecture

Email addresses are captured client-side and sent out-of-band rather than being blocked at the server level. This allows the real address to be used for the confirmation email and Supabase record, while ensuring the LLM provider never receives it. The alternative (masking at the server but never restoring) would have required a separate mechanism to collect the email outside the chat flow entirely.

14.5 Phase-Driven Step Tracker vs. Message Count

An early implementation tracked progress by counting the number of messages exchanged. This was replaced with the phase-tag approach because message count is unreliable (the AI might take more or fewer turns to troubleshoot) whereas the phase tag is explicitly emitted by the model at a semantically meaningful point in the conversation.

15 Challenges and Lessons Learned

15.1 Prompt Engineering Stability

Getting the model to reliably emit exactly one `<PHASE>` tag per response, in the correct position, required several iterations of the system prompt. The key insight was to give the model an explicit, rule-labelled section ("**RULES**") stating the tag requirement, rather than burying it in prose instructions. Structured prompt sections with visual separators () improved compliance significantly.

15.2 Environment Variable Formatting

Inline comments in `.env` files (`KEY=value # comment`) are not stripped by all `dotenv` implementations. The comment became part of the key value, causing authentication failures that were non-obvious to debug. The lesson: never write inline comments in `.env` files; put explanatory text in a separate `.env.example` file.

15.3 Vercel vs. AWS Lambda Configuration

Vercel's `vercel.json` has a different schema from AWS Lambda function configuration. The `"runtime"` key is specific to Lambda and is rejected by Vercel. Vercel auto-detects the runtime from `package.json`, so no explicit runtime declaration is needed.

15.4 iOS Safari Input Zoom

iOS Safari auto-zooms any text input with a computed `font-size` below `16px` when the input receives focus. This caused the chat layout to shift jarringly on mobile. The fix is simple (`font-size: 16px` on the input element) but easy to miss if you do not test on a real iOS device.

15.5 Honest Metrics

An early version of the landing page included specific performance figures. These were revised to qualitative, representative language with an explicit disclaimer, for two reasons: (1) the specific figures could not be publicly verified without disclosing internal data, and (2) interviewers are more likely to trust honest framing than precise numbers attached to a demo project.

16 Conclusion

This project demonstrates end-to-end ownership of an AI automation system — from problem framing and architecture design, through full-stack implementation, to production deployment and observability. The demo faithfully reproduces the conversational UX and data flow of a production enterprise system using freely available tooling, with no proprietary assets.

The core technical contributions are:

- A two-phase AI conversation protocol (troubleshoot-first, escalate-on-failure) encoded in a structured system prompt and enforced via phase tags.
- A privacy layer that redacts PII before any message leaves the server toward a third-party AI provider, while preserving the real data for legitimate use (ticket storage and email).
- A stateless serverless backend that integrates three external services (Groq, Supabase, Resend) within a single handler, with graceful fallback if any service is unconfigured.
- A phase-driven frontend state machine that synchronises UI progress, quick-reply options, and input locking with the AI's conversation state.

The live demo is accessible at:

<https://ryco-ai-support-demo.vercel.app>

A Dependencies

Package	Version	Purpose
groq-sdk	^0.15.0	Groq API client (Llama 3.3 70B)
@supabase/supabase-js	^2.49.0	Supabase PostgreSQL client
resend	^4.5.0	Transactional email API client

Runtime requirement: Node.js ≥ 18 (specified in `package.json` engines).

B Groq Model Selection

The model used is **Llama 3.3 70B Versatile** (llama-3.3-70b-versatile) via Groq's inference API. This was selected because:

- Available on Groq's free tier with generous rate limits.
- Sufficient instruction-following capability to reliably emit structured tags.
- Fast inference (typically 1–3 seconds) due to Groq's LPU hardware.
- No data training on prompts under the free tier (per Groq's privacy policy at time of implementation).

Maximum token budget per response is set to 512 tokens, sufficient for an 80-word prose response plus the structured JSON block.

C Full System Prompt

```
1 You are an IT Support Agent for RYCO. You help employees resolve
2 IT issues through a two-phase approach: first try to fix it
3 yourself, then escalate if needed.
4
5 *** PHASE 1 - TROUBLESHOOT ***
6 When a user first describes an IT issue:
7 - Acknowledge it in one sentence
8 - Give 1-2 specific, actionable troubleshooting steps tailored
9   to their exact problem
10 - Ask them to try and report back
11 - End your message with: <PHASE>helping</PHASE>
12
13 If the user says it WORKED -> congratulate them briefly, no
14 ticket needed. End with: <PHASE>resolved</PHASE>
15
16 If the user says it DIDN'T WORK -> try a different approach
17 (do NOT repeat the same suggestion).
18 End with: <PHASE>helping</PHASE>
19
20 After 3 failed troubleshooting attempts, OR if the user says
21 "just create a ticket" / "escalate" / "I give up":
22 - Acknowledge the issue persists, tell them you'll escalate
23 - Ask for their work email address
24 - End with: <PHASE>asking_email</PHASE>
```

```
25
26 *** PHASE 2 - ESCALATE ***
27 Once you have a valid email address:
28 - Confirm you have it
29 - Ask how urgent this is, listing three options on separate lines:
30   High - need it resolved immediately
31   Medium - within the next hour
32   Low - today works
33 - End with: <PHASE>asking_urgency</PHASE>
34
35 Once the user selects urgency:
36 - Confirm you're creating the ticket
37 - End with <PHASE>creating_ticket</PHASE> then the ticket block
38
39 *** RULES ***
40 - Keep every response under 80 words
41 - Be specific with troubleshooting - tailor to what user said
42 - Never suggest the same fix twice
43 - Never ask for email before troubleshooting has failed
44 - Always end every response with exactly one <PHASE> tag
45 - <PHASE> and <TICKET_DATA> tags are stripped before user sees
   them
46
47 *** PRIVACY ***
48 Emails are replaced with [REDACTED_EMAIL] before reaching you.
49 Use [REDACTED_EMAIL] in ticket JSON - server restores real
   address.
50
51 *** CATEGORY / TEAM / SLA ***
52 Network / VPN -> VPN, WiFi, connectivity -> Network Operations ->
   30 min
53 Access / Permissions -> password, MFA, locked out -> Identity &
   Access -> 30 min
54 Hardware -> laptop, monitor, keyboard, printer -> Hardware
   Support -> 2 hours
55 Software / M365 -> Outlook, Teams, Office, M365 -> Software
   Support -> 1 hour
56 Software -> app, crash, install, slow -> Software Support -> 1
   hour
57 General -> anything else -> IT Support -> 2 hours
58
59 *** TICKET DATA FORMAT ***
60 <TICKET_DATA>
61 {
62   "email": "[REDACTED_EMAIL]",
63   "issue_summary": "one-line title",
64   "details": "description including troubleshooting attempted",
65   "urgency": "High|Medium|Low",
66   "category": "Network / VPN|Access / Permissions|Hardware|
67               Software / M365|Software|General"
68 }
```

```
69 </TICKET_DATA>
```

Listing 21: Complete system prompt (api/chat.js)